



Universidad Europea

UNIVERSIDAD EUROPEA DE MADRID
ESCUELA DE ARQUITECTURA, INGENIERÍA Y DISEÑO
GRADO EN INGENIERÍA INFORMÁTICA

MEMORIA COMPILADOR

COMPILADORES Y LENGUAJES FORMALES

ÁLVARO CARRIZOSA PEÑA

GUILLERMO AOS RODERA

Dirigido por:

LEOPOLDO SANTOS SANTOS

CURSO 2024-2025

Álvaro Carrizosa Peña

Guillermo Aós Rodera

Memoria Compilador

1. Introducción al problema	3
Objetivos Principales	3
Características Implementadas	3
2. Solución propuesta	4
2.1 Frontend del Compilador	4
ruby_lexer.flex - Analizador Léxico	4
ruby_parser.y - Analizador Sintáctico	5
AST_ruby.h/c - Árbol de Sintaxis Abstracta	5
2.2 Gestión de Símbolos	5
tabla_simbolos.h/c - Tabla de Variables c	5
2.3 Backend del Compilador	6
2.3.1 generar_mips_completo.c - Generador de Código	6
2.3.2 Flujo de Compilación Detallado	7
ruby_lexer.flex	8
ruby_parser.y	8
tabla_simbolos.h	9
tabla_simbolos.c	9
tabla_simbolos_funciones.h	10
tabla_simbolos_funciones.c	11
AST_ruby.h	11
AST_ruby.c	13
generar_mips_completo.c	17
3. Directivas de compilación	17
3.1 Requisitos del Sistema	17
3.2 Instalación del Entorno	17
3.3 Estructura de Directorios del Proyecto	18
4. Manual del usuario	19
4.1 Introducción	19
4.2 Ejecución del Compilador	19
4.2.1 Uso Básico	19
4.2.2 Opciones de Línea de Comandos	20
4.3. Archivos de Salida	20
4.4 Ejecución en MARS	20
4.4.1 Método 1: Interfaz Gráfica	20
4.5 Sintaxis Ruby Soportada	21
4.5.1 Tipos de Datos	21
4.5.2 Operaciones Aritméticas	21
4.5.3 Operaciones de Comparación	21
4.5.4 Operaciones Lógicas	21
5. Batería de pruebas	22
5.1 Prueba 1	22
5.1 Prueba 2	25
5.1 Prueba 3	27
6. Comentarios y conclusiones	30

1.Introducción al problema

Trabajo Colaborativo:

- Sabemos que había que poner exactamente quién había hecho cada cosa del compilador, pero la verdad que todo ha sido bastante colaborativo, no ha habido ningún elemento en el que haya participado sólo uno de nosotros.

Este proyecto implementa un compilador completo que traduce código Ruby a ensamblador MIPS, ejecutable en el simulador MARS. El compilador está desarrollado en C.

Objetivos Principales

1. Análisis Léxico: Reconocimiento de tokens del lenguaje Ruby
2. Análisis Sintáctico: Construcción del Árbol de Sintaxis Abstracta (AST)
3. Análisis Semántico: Verificación de tipos y validación semántica
4. Generación de Código: Traducción a ensamblador MIPS optimizado
5. Gestión de Memoria: Tabla de símbolos y gestión de variables

Características Implementadas

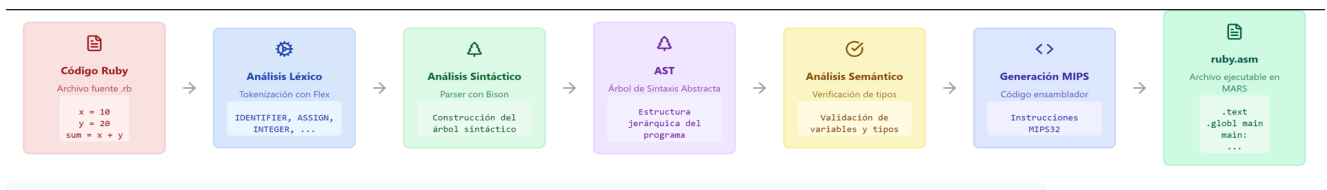
- Tipos de Datos: Integer, Float, String, Boolean
- Operadores: Aritméticos (+, -, *, /), Booleanos (and, or, not), Comparación
- Control de Flujo: if-else-elsif, while, for

Álvaro Carrizosa Peña
Guillermo Aós Rodera
Memoria Compilador

- Funciones: Definición, llamadas, parámetros, return
- Arrays: Arrays 1D y 2D con indexación
- E/S: puts para salida de datos

2. Solución propuesta

Arquitectura del Compilador: El compilador sigue la arquitectura clásica de compiladores con las siguientes fases claramente diferenciadas:



Proceso de Compilación:

1. Análisis Léxico: Flex genera yylex() para tokenización
2. Análisis Sintáctico: Bison genera yyparse() para parsing
3. Compilación: GCC enlaza archivos fuente y librerías
4. Generación: Produce código MIPS ejecutable en MARS

2.1 Frontend del Compilador

ruby_lexer.flex - Analizador Léxico

- Utiliza Flex para generar un analizador léxico determinista
- Reconoce tokens mediante expresiones regulares
- Maneja comentarios, espacios en blanco y conteo de líneas
- Genera valores semánticos para literales numéricos y strings

ruby_parser.y - Analizador Sintáctico

- Implementa un parser LALR(1) usando Bison
- Define gramática libre de contexto para Ruby
- Construye AST durante el análisis sintáctico
- Maneja precedencia de operadores y recuperación de errores

AST_ruby.h/c - Árbol de Sintaxis Abstracta

- Estructura de datos jerárquica para representar el programa
- Nodos para expresiones, sentencias y declaraciones
- Funciones de construcción y manipulación del árbol
- Soporte para recorrido y generación de código

2.2 Gestión de Símbolos

tabla_simbolos.h/c - Tabla de Variables c

```
typedef struct {  
    char* nombre;           // Nombre del identificador  
    char* tipo;            // Tipo: "integer", "float", "string", "boolean"  
    int integer;           // Valor si es entero  
    float floatDecimal;    // Valor si es float  
}
```

```
char* string;      // Valor si es string  
int boolean;      // Valor si es boolean  
int registro;     // Registro MIPS asignado  
int direccion;    // Dirección en memoria  
int inicializada; // Estado de inicialización  
} tSimbolos;
```

tabla_simbolos_funciones.h/c

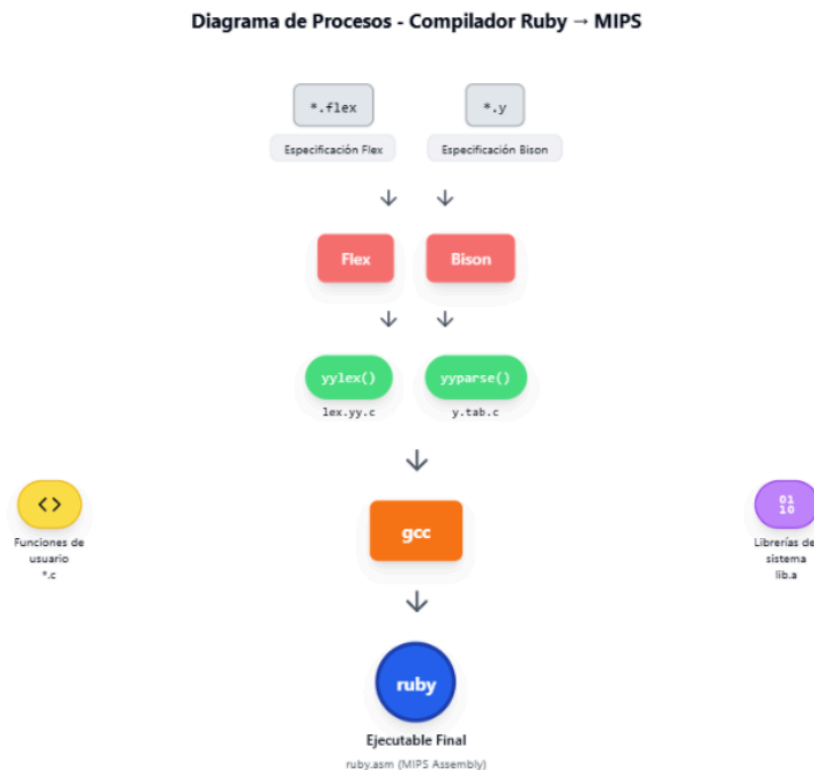
- Gestión de Funciones
- Tabla separada para funciones definidas por el usuario
- Gestión de parámetros y tipos de retorno
- Validación de llamadas a funciones
- Soporte para recursión mediante gestión de pila

2.3 Backend del Compilador

2.3.1 generar_mips_completo.c - Generador de Código

- Traduce AST a instrucciones MIPS32
- Gestión automática de registros temporales (\$t0-\$t7)
- Optimizaciones básicas de código
- Generación de prólogos y epílogos de funciones

2.3.2 Flujo de Compilación Detallado



1. Preprocesamiento: Lectura y validación del archivo fuente
2. Análisis Léxico: Tokenización usando ruby_lexer.flex → lex.yy.c
3. Análisis Sintáctico: Parsing usando ruby_parser.y → ruby_parser.tab.c
4. Construcción AST: Creación del árbol durante el parsing
5. Análisis Semántico: Verificación de tipos y validación de variables
6. Optimización: Aplicación de optimizaciones básicas
7. Generación de Código: Traducción AST → MIPS32
8. Salida: Archivo ruby.asm ejecutable en MARS

Decisiones de Diseño

Gestión de Memoria:

- Uso de malloc/free para gestión dinámica
- Liberación automática de memoria del AST tras generación
- Tabla de símbolos con tamaño fijo para simplificar gestión

Álvaro Carrizosa Peña
Guillermo Aós Rodera
Memoria Compilador

Asignación de Registros:

- Estrategia simple: variables frecuentes en registros temporales
- Uso de convenciones MIPS para llamadas a funciones

Manejo de Errores:

- Recuperación de errores sintácticos mediante reglas de error
- Reportes detallados con números de línea
- Continuación de análisis tras errores no fatales

`ruby_lexer.flex`

Definición del analizador léxico con Flex

Función	Descripción
<code>main()</code>	Función principal generada automáticamente por Flex que controla el proceso de tokenización
<code>yylex()</code>	Función principal del analizador léxico que lee caracteres y devuelve tokens
<code>yywrap()</code>	Función que indica si hay más archivos para procesar (deshabilitada con <code>%option noyywrap</code>)

`ruby_parser.y`

Gramática del compilador con Bison

Función	Descripción
<code>main()</code>	Función principal que abre archivos, inicializa el parser y controla la compilación completa

<code>yyparse()</code>	Función principal del parser que analiza la sintaxis y construye el AST
<code>yyerror()</code>	Maneja errores de sintaxis e imprime mensajes de error con número de línea
<code>agregarParametrosATabla()</code>	Agrega parámetros de funciones a la tabla de símbolos durante la definición de funciones

tabla_simbolos.h

Función	Descripción
<code>buscarTabla()</code>	Declaración de función para buscar símbolos en la tabla
<code>agregarSimbolo()</code>	Declaración de función para agregar nuevos símbolos
<code>inicializarTabla()</code>	Declaración de función para inicializar la tabla
<code>imprimirTabla()</code>	Declaración de función para mostrar el contenido de la tabla
<code>obtenerRegistro()</code>	Declaración de función para obtener registro MIPS asignado
<code>liberarTabla()</code>	Declaración de función para liberar memoria de la tabla

tabla_simbolos.c

Función	Descripción
inicializarTabla()	Inicializa todos los campos de la tabla de símbolos a valores por defecto (NULL, 0)
buscarTabla()	Busca una variable por nombre en la tabla y devuelve su índice
agregarSimbolo()	Agrega una nueva variable a la tabla con nombre, tipo y registro MIPS asignado
obtenerRegistro()	Devuelve el nombre del registro MIPS (\$t0-\$t7) correspondiente a una posición
imprimirTabla()	Imprime el contenido completo de la tabla de símbolos en formato tabular
liberarTabla()	Libera toda la memoria dinámica asignada para nombres, tipos y strings
duplicar_string()	Función auxiliar local para duplicar cadenas de caracteres con malloc

tabla_simbolos_funciones.h

Header para la tabla de funciones

Función	Descripción
agregarFuncion()	Declaración de función para agregar funciones a la tabla

<code>buscarFuncion()</code>	Declaración de función para buscar funciones por nombre
------------------------------	---

tabla_simbolos_funciones.c

Función	Descripción
<code>agregarFuncion()</code>	Agrega una nueva función a la tabla con nombre, parámetros y cuerpo del AST
<code>buscarFuncion()</code>	Busca una función por nombre y devuelve su índice en la tabla

AST_ruby.h

Header del Árbol de Sintaxis Abstracta

Función	Descripción
<code>crearNodoTerminal()</code>	Declaración para crear nodos terminales con valores enteros
<code>crearNodoTerminalFloat()</code>	Declaración para crear nodos terminales con valores float
<code>crearNodoTerminalString()</code>	Declaración para crear nodos terminales con strings
<code>crearNodoTerminalBool()</code>	Declaración para crear nodos terminales booleanos
<code>crearVariableTerminal()</code>	Declaración para crear nodos de variables con registro asignado

<code>crearNodoNoTerminal()</code>	Declaración para crear nodos de operaciones binarias
<code>crearNodoUnario()</code>	Declaración para crear nodos de operaciones unarias
<code>crearNodoAsignacion()</code>	Declaración para crear nodos de asignación de variables
<code>crearNodoAsignacionCompuesta()</code>	Declaración para crear nodos de asignación compuesta (<code>+=</code> , <code>-=</code>)
<code>crearNodoPuts()</code>	Declaración para crear nodos de impresión
<code>crearNodoCondicional()</code>	Declaración para crear nodos if-else
<code>crearNodoBucle()</code>	Declaración para crear nodos de bucles while
<code>crearNodoVacio()</code>	Declaración para crear nodos vacíos (líneas en blanco)
<code>crearNodoFuncion()</code>	Declaración para crear nodos de definición de funciones
<code>crearNodoLlamadaFuncion()</code>	Declaración para crear nodos de llamadas a funciones
<code>crearNodoParametro()</code>	Declaración para crear nodos de parámetros de funciones
<code>crearNodoAccesoArray()</code>	Declaración para crear nodos de acceso a arrays 2D

<code>crearNodoAsignacionArray()</code>	Declaración para crear nodos de asignación a arrays 2D
<code>liberarAST()</code>	Declaración para liberar memoria del AST recursivamente
<code>imprimirAST()</code>	Declaración para imprimir estructura del AST
<code>comprobarAST()</code>	Declaración para validar y procesar el AST completo
<code>generarCodigo()</code>	Declaración para generar código (función de compatibilidad)
<code>generarCodigoMIPS()</code>	Declaración para generar código MIPS por consola
<code>generarCodigoMIPSArchivo()</code>	Declaración para generar código MIPS en archivo
<code>generarMIPSCompleto()</code>	Declaración para generar archivo MIPS completo ejecutable
<code>obtenerNuevoRegistro()</code>	Declaración para asignar registros MIPS automáticamente
<code>obtenerNuevaEtiqueta()</code>	Declaración para generar etiquetas únicas para saltos

AST_ruby.c

Implementación completa del Árbol de Sintaxis Abstracta

Función	Descripción
---------	-------------

<code>crearNodoTerminal()</code>	Crea nodos terminales para valores enteros con asignación automática de registro
<code>crearNodoTerminalFloat()</code>	Crea nodos terminales para valores float con asignación automática de registro
<code>crearNodoTerminalString()</code>	Crea nodos terminales para strings duplicando la cadena en memoria
<code>crearNodoTerminalBool()</code>	Crea nodos terminales para valores booleanos (0 o 1)
<code>crearVariableTerminal()</code>	Crea nodos para variables con registro específico asignado
<code>crearNodoNoTerminal()</code>	Crea nodos para operaciones binarias con dos hijos
<code>crearNodoUnario()</code>	Crea nodos para operaciones unarias con un solo hijo
<code>crearNodoAsignacion()</code>	Crea nodos de asignación simple (variable = expresión)
<code>crearNodoAsignacionCompuesta()</code>	Crea nodos de asignación compuesta (+=, -=) con operación específica
<code>crearNodoPuts()</code>	Crea nodos para sentencias de impresión (puts)
<code>crearNodoCondicional()</code>	Crea nodos if-else con condición, bloque if y bloque else opcional

<code>crearNodoBucle()</code>	Crea nodos de bucles while con condición y cuerpo
<code>crearNodoVacio()</code>	Crea nodos vacíos para líneas en blanco o sentencias nulas
<code>crearNodoFuncion()</code>	Crea nodos de definición de funciones con nombre, parámetros y cuerpo
<code>crearNodoLlamadaFuncion()</code>	Crea nodos de llamadas a funciones con argumentos
<code>crearNodoParametro()</code>	Crea nodos individuales para parámetros de funciones
<code>crearNodoAccesoArray()</code>	Crea nodos para acceso a elementos de arrays bidimensionales
<code>crearNodoAsignacionArray()</code>	Crea nodos para asignación a elementos de arrays bidimensionales
<code>obtenerNuevoRegistro()</code>	Asigna automáticamente registros MIPS (\$t0-\$t7) de forma circular
<code>obtenerNuevaEtiqueta()</code>	Genera etiquetas únicas (L0, L1, L2...) para saltos condicionales
<code>liberarAST()</code>	Libera recursivamente toda la memoria del AST incluyendo strings y etiquetas
<code>imprimirAST()</code>	Imprime la estructura del AST de forma jerárquica con indentación

<code>comprobarAST()</code>	Valida el AST y lanza la generación de código MIPS completo
<code>esDefinicionFuncion()</code>	Verifica si un nodo es una definición de función
<code>obtenerRegistroParametro()</code>	Obtiene el registro asignado a un parámetro específico por nombre
<code>asegurarRegistroParametro()</code>	Asegura que un nodo use el registro correcto para parámetros
<code>generarCodigoFuncion()</code>	Genera código MIPS específico para el contexto de funciones con parámetros
<code>generarCodigoPrincipal()</code>	Genera código MIPS para el programa principal excluyendo definiciones de funciones
<code>generarDefinicionesFunciones()</code>	Genera todas las definiciones de funciones con prólogo y epílogo completos
<code>generarCodigoMIPSArchivo()</code>	Coordina la generación completa de código MIPS en archivo
<code>generarCodigoMIPS()</code>	Genera código MIPS por consola (función mejorada para funciones)
<code>generarCodigo()</code>	Función vacía para compatibilidad con versiones anteriores

generar_mips_completo.c

Generador completo de código MIPS ejecutable

Función	Tipo	Parámetros	Retorna	Descripción
generarMIPSCompleto()	Principal	struct ast* nodo, const char* nombreArchivo	void	Función principal que genera un archivo ruby.asm completo y ejecutable en MARS con secciones .data y .text

3. Directivas de compilación

Esta sección proporciona instrucciones detalladas para configurar el entorno de desarrollo y compilar el proyecto en Windows con MinGW-w64.

3.1 Requisitos del Sistema

Software Requerido:

- MinGW-w64 versión 8.1.0 o superior
- MSYS2 para gestión de paquetes
- MARS 4.5 (MIPS Assembly and Runtime Simulator)
- Editor de texto (recomendado: VS Code, Notepad++)

3.2 Instalación del Entorno

Paso 1: Verificación de la Instalación

Ejecutar en CMD o PowerShell:

```
gcc --version      # Debe mostrar gcc (Rev10, Built by MSYS2 project) 11.2.0
```

```
flex --version      # Debe mostrar flex 2.6.4  
bison --version     # Debe mostrar bison (GNU Bison) 3.8.2  
make --version      # Debe mostrar GNU Make 4.3
```

Paso 2: Instalación de MARS

1. Descargar MARS:

- Ir a <http://courses.missouristate.edu/KenVollmar/mars/>
- Descargar Mars4_5.jar
- Colocar en directorio de trabajo del proyecto

3.3 Estructura de Directorios del Proyecto

Ruta / Archivo	Descripción
src/AST_ruby.c	Implementación del Árbol de Sintaxis Abstracta (AST)
src/AST_ruby.h	Definiciones del AST
src/generar_mips_completo.c	Generador de código MIPS completo
src/Makefile	Script de compilación
src/ruby_lexer.flex	Análisis léxico usando Flex
src/ruby_parser.y	Análisis sintáctico con Bison
src/tabla_simbolos.c	Tabla de símbolos (variables)
src/tabla_simbolos.h	Cabecera de la tabla de símbolos
src/tabla_simbolos_funciones.c	Tabla de símbolos (funciones)
src/tabla_simbolos_funciones.h	Cabecera de la tabla de símbolos de funciones
ejecutable/ruby_compiler.exe	Ejecutable del compilador
ejecutable/lex.yy.c	Código C generado por Flex

<code>ejecutable/ruby_parser.tab.c</code>	Código C generado por Bison
<code>ejecutable/ruby_parser.tab.h</code>	Cabeceras generadas por Bison
<code>ejecutable/Mars4_5.jar</code>	Simulador MARS para código ensamblador MIPS
<code>ejecutable/ruby.asm</code>	Código ensamblador MIPS generado
<code>pruebas/test.rb</code>	Programa de prueba básico en Ruby
<code>pruebas/test_control.rb</code>	Pruebas de estructuras de control
<code>pruebas/test_funciones.rb</code>	Pruebas de funciones
<code>MEMORIA_COMPILADOR.pdf</code>	Documento de memoria del proyecto del compilador

4. Manual del usuario

4.1 Introducción

Este manual proporciona las instrucciones necesarias para ejecutar correctamente el compilador de Ruby a

MIPS desarrollado en el proyecto. Se incluyen ejemplos de uso, descripción de archivos generados, ejecución en el simulador MARS y sintaxis Ruby soportada.

4.2 Ejecución del Compilador

4.2.1 Uso Básico

```
#Compilar el compilador
make

# Sintaxis general:

    ruby_compiler.exe <archivo_fuente.rb>

# Ejemplo práctico:
```

```
c./ejecutable/ruby_compiler.exe  
pruebas/test_control.rb
```

4.2.2 Opciones de Línea de Comandos

```
# Compilar con información de debug  
ruby_compiler.exe -v ../pruebas/test.rb  
  
# Compilar mostrando tabla de símbolos  
ruby_compiler.exe -s ../pruebas/test.rb  
  
# Compilar con análisis de optimización  
ruby_compiler.exe -O ../pruebas/test.rb
```

4.3. Archivos de Salida

- ruby.asm: Código MIPS generado, compatible con MARS.

4.4 Ejecución en MARS

4.4.1 Método 1: Interfaz Gráfica

1. Ejecutar `java -jar Mars4_5.jar`
2. File -> Open -> Seleccionar `ruby.asm`
3. Run -> Assemble (F3)
4. Run -> Go (F5)
5. Verificar salida en consola

4.5 Sintaxis Ruby Soportada

4.5.1 Tipos de Datos

```
edad = 25  
pi = 3.14  
nombre = "Juan"  
activo = true
```

4.5.2 Operaciones Aritméticas

```
suma = a + b  
contador += 1  
precio *= 1.21
```

4.5.3 Operaciones de Comparación

```
mayor = edad > 18  
igual = resultado == esperado  
nombre_igual = nombre == "Pedro"
```

4.5.4 Operaciones Lógicas

```
resultado_and = (a > 5) and (b < 10)
```

```
negacion = not terminado
```

```
acceso_permitido = (usuario_valido and password_correcta)  
and not bloqueado
```

5. Batería de pruebas

5.1 Prueba 1

Para la primera prueba, haremos uso de nuestro archivo llamado “test.rb” (.rb es la extensión de los archivos en lenguaje Ruby). El contenido de dicho archivo es el siguiente:

```
# Archivo de prueba básico para el compilador Ruby -> MIPS
```

```
# Operaciones aritméticas básicas
```

```
x = 10
```

```
y = 20
```

```
z = x + y
```

```
puts z
```

```
# Operaciones con diferentes tipos
```

```
a = 5
```

```
b = 3
```

```
resultado = a * b - 2
```

```
puts resultado
```

Tras ejecutar todo lo necesario con el comando “make”, el cual ejecuta todo lo necesario para tener nuestro compilador funcional, ejecutamos este archivo con el comando:

```
./ejecutable/ruby_compiler.exe pruebas/test.rb
```

Se compila exitosamente como podemos ver en pantalla, además de identificar todos los símbolos y crear el AST apropiado:

```
alvar@PC_ALVARO ~
$ ./ejecutable/ruby_compiler.exe pruebas/test.rb
Iniciando compilacion de pruebas/test.rb...
> [TIPO] - INTEGER: 10
> [SENTENCIA] - Asignacion Simple
Asignado el valor 10 a la variable x
> [TIPO] - INTEGER: 20
> [SENTENCIA] - Asignacion Simple
Asignado el valor 20 a la variable y
> [TIPO] - IDENTIFICADOR: x
> [TIPO] - IDENTIFICADOR: y
> [OPERACION] - SUMA {integer + integer}
> [SENTENCIA] - Asignacion Simple
Asignado el valor 30 a la variable z
> [TIPO] - IDENTIFICADOR: z
> [SENTENCIA] - PUTS
> [TIPO] - INTEGER: 5
> [SENTENCIA] - Asignacion Simple
Asignado el valor 5 a la variable a
> [TIPO] - INTEGER: 3
> [SENTENCIA] - Asignacion Simple
Asignado el valor 3 a la variable b
> [TIPO] - IDENTIFICADOR: a
> [TIPO] - IDENTIFICADOR: b
> [OPERACION] - MULTIPLICACION {integer * integer}
> [TIPO] - INTEGER: 2
> [OPERACION] - RESTA {integer - integer}
> [SENTENCIA] - Asignacion Simple
Asignado el valor 13 a la variable resultado
> [TIPO] - IDENTIFICADOR: resultado
> [SENTENCIA] - PUTS

=== GENERANDO CÓDIGO MIPS COMPLETO ===

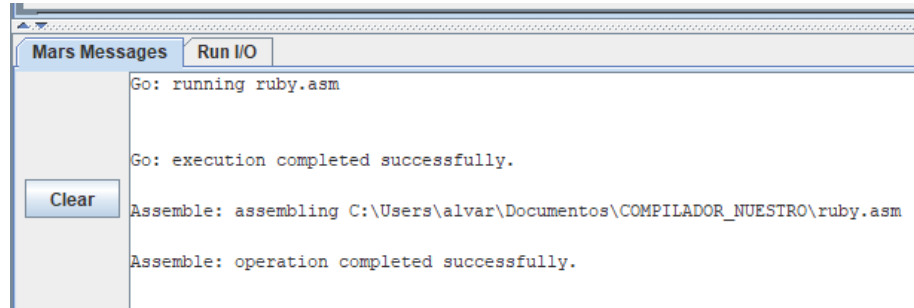
=== CODIGO MIPS GENERADO EN ruby.asm ===
```

```
        SUMA
        NODO_TIPO_34
        NODO_TIPO_34
    PUTS
        NODO_TIPO_34
        NODO_TIPO_25
        NODO_TIPO_25
        ASIGNACION: a
        NUMERO: 5
        ASIGNACION: b
        NUMERO: 3
        ASIGNACION: resultado
        RESTA
        MULTIPLICACION
        NODO_TIPO_34
        NODO_TIPO_34
        NUMERO: 2
    PUTS
        NODO_TIPO_34

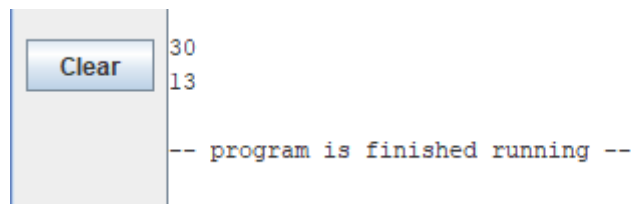
=== TABLA DE SÍMBOLOS ===
Nombre      Tipo      Valor      Registro
=====
x            integer    10         $t2
y            integer    20         $t4
z            integer    30         $t6
a            integer    5          $t3
b            integer    3          $t5
resultado    integer    13         $t1

[COMPILACION FINALIZADA]
Compilacion exitosa. Codigo MIPS generado en ruby.asm
```

Tras esto, nos vamos al MARS, abrimos el archivo “ruby.asm” tal y como hemos explicado en el apartado anterior(4), lo compilamos, y vemos como compila exitosamente:



Ahora, lo ejecutamos, y este es nuestro resultado:



5.1 Prueba 2

Para la segunda prueba, haremos uso de nuestro archivo llamado "test_control.rb" (.rb es la extensión de los archivos en lenguaje Ruby). El contenido de dicho archivo es el siguiente:

```
# Archivo de prueba para estructuras de control
# If/else y while
numero = 15

if numero > 10
  puts "El número es mayor que 10"
else
  puts "El número es menor o igual que 10"
end

contador = 0
while contador < 5
  puts contador
  contador = contador + 1
end
```

Tras ejecutar todo lo necesario con el comando "make", el cual ejecuta todo lo necesario para tener nuestro compilador funcional, ejecutamos este archivo con el comando:

```
./ejecutable/ruby_compiler.exe pruebas/test.rb
```

Se compila exitosamente como podemos ver en pantalla, además de identificar todos los símbolos y crear el AST apropiado:

```
alvar@PC_ALVARO ~
$ ./ejecutable/ruby_compiler.exe pruebas/test_control.rb
Iniciando compilación de pruebas/test_control.rb...
> [TIPO] - INTEGER: 15
> [SENTENCIA] - Asignacion Simple
Asignado el valor 15 a la variable numero
> [TIPO] - IDENTIFICADOR: numero
> [TIPO] - INTEGER: 10
> [OPERACION] - MAYOR QUE
> [TIPO] - STRING: "El número es mayor que 10"
> [SENTENCIA] - PUTS
> [TIPO] - STRING: "El número es menor o igual que 10"
> [SENTENCIA] - PUTS
> [SENTENCIA] - IF-ELSE
> [TIPO] - INTEGER: 0
> [SENTENCIA] - Asignacion Simple
Asignado el valor 0 a la variable contador
> [TIPO] - IDENTIFICADOR: contador
> [TIPO] - INTEGER: 5
> [OPERACION] - MENOR QUE
> [TIPO] - IDENTIFICADOR: contador
> [SENTENCIA] - PUTS
> [TIPO] - IDENTIFICADOR: contador
> [TIPO] - INTEGER: 1
> [OPERACION] - SUMA {integer + integer}
> [SENTENCIA] - Asignacion Simple
Asignado el valor 1 a la variable contador
> [SENTENCIA] - WHILE

=== GENERANDO CÓDIGO MIPS COMPLETO ===

=== CÓDIGO MIPS GENERADO EN ruby.asm ===
El archivo ruby.asm ha sido creado exitosamente.
Puedes ejecutarlo en MARS (MIPS Assembly and Runtime Simulator)

=== AST CONSTRUIDO CORRECTAMENTE ===
NODO_TIPO_24
```

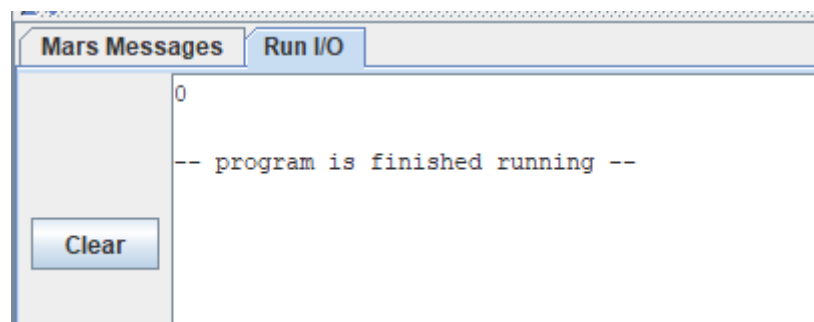
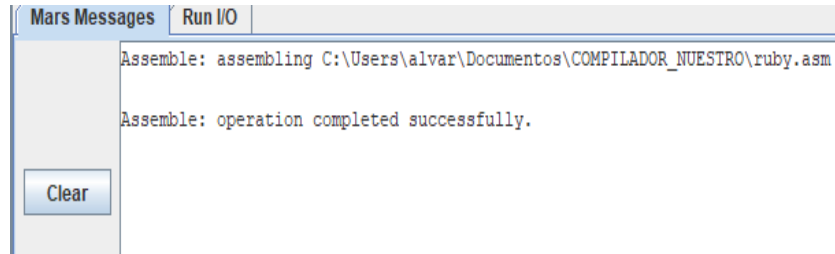
```
NODO_TIPO_25
ASIGNACION: contador
NUMERO: 0
NODO_TIPO_23
NODO_TIPO_10
NODO_TIPO_34
NUMERO: 5
NODO_TIPO_24
PUTS
NODO_TIPO_34
ASIGNACION: contador
SUMA
NODO_TIPO_34
NUMERO: 1

=== TABLA DE SÍMBOLOS ===
Nombre      Tipo      Valor      Registro
=====
numero      integer   15         $t2
contador    integer   0          $t3
contador    integer   1          $t0

[COMPILACION FINALIZADA]
Compilacion exitosa. Codigo MIPS generado en ruby.asm
```

Álvaro Carrizosa Peña
Guillermo Aos Rodera
Memoria Compilador

Tras esto, nos vamos al MARS, abrimos el archivo "ruby.asm" tal y como hemos explicado en el apartado anterior(4), lo compilamos, y vemos como compila exitosamente:



5.1 Prueba 3

Para la tercera prueba, haremos uso de nuestro archivo llamado "test_funciones.rb" (.rb es la extensión de los archivos en lenguaje Ruby). El contenido de dicho archivo es el siguiente:

```
# Archivo de prueba para funciones

def suma(a, b)
  return a + b
end

def factorial(n)
  if n <= 1
    return 1
  else
    return n * factorial(n - 1)
  end
end
```

```

        end

    end

# Llamadas a funciones
resultado1 = suma(5, 3)

puts resultado1

resultado2 = factorial(4)

puts resultado2

```

Tras ejecutar todo lo necesario con el comando “make”, el cual ejecuta todo lo necesario para tener nuestro compilador funcional, ejecutamos este archivo con el comando:

```
./ejecutable/ruby_compiler.exe pruebas/test.rb
```

Se compila exitosamente como podemos ver en pantalla, además de identificar todos los símbolos y crear el AST apropiado:

```

alvar@PC_ALVARO ~
$ ./ejecutable/ruby_compiler.exe pruebas/test_funciones.rb
Iniciando compilacion de pruebas/test_funciones.rb...
> [PARAMETRO] - a
> [PARAMETRO] - b
> [FUNCION] - Definicion de funcion con parametros: suma
Funcion 'suma' agregada a la tabla
Agregando parametro 'b' a la tabla de simbolos
Agregando parametro 'a' a la tabla de simbolos
> [TIPO] - IDENTIFICADOR: a
> [TIPO] - IDENTIFICADOR: b
> [OPERACION] - SUMA {integer + integer}
> [SENTENCIA] - RETURN
> [PARAMETRO] - n
> [FUNCION] - Definicion de funcion con parametros: factorial
Funcion 'factorial' agregada a la tabla
Agregando parametro 'n' a la tabla de simbolos
> [TIPO] - IDENTIFICADOR: n
> [TIPO] - INTEGER: 1
> [OPERACION] - MENOR O IGUAL QUE
> [TIPO] - INTEGER: 1
> [SENTENCIA] - RETURN
> [TIPO] - IDENTIFICADOR: n
> [TIPO] - IDENTIFICADOR: n
> [TIPO] - INTEGER: 1
> [OPERACION] - RESTA {integer - integer}
> [LLAMADA] - Llamada a funcion: factorial
> [OPERACION] - MULTIPLICACION {integer * integer}
> [SENTENCIA] - RETURN
> [SENTENCIA] - IF-ELSE
> [TIPO] - INTEGER: 5

```

Álvaro Carrizosa Peña
Guillermo Aos Rodera
Memoria Compilador

```

MULTIPLICACION
NODO_TIPO_34
LLAMADA_FUNCION: factorial
RESTA
NODO_TIPO_34
NUMERO: 1

NODO_TIPO_25
NODO_TIPO_25
ASIGNACION: resultado1
LLAMADA_FUNCION: suma
NUMERO: 3
NUMERO: 5

PUTS
NODO_TIPO_34
NODO_TIPO_25
ASIGNACION: resultado2
LLAMADA_FUNCION: factorial
NUMERO: 4

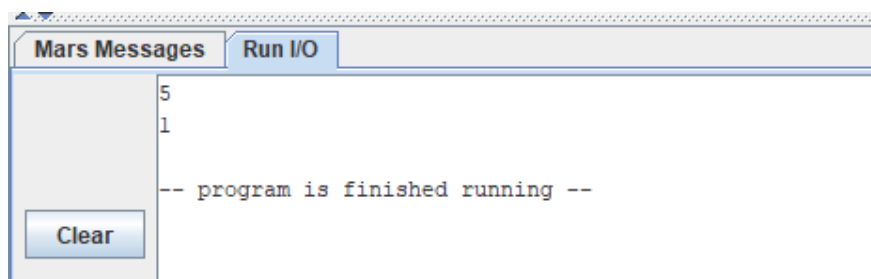
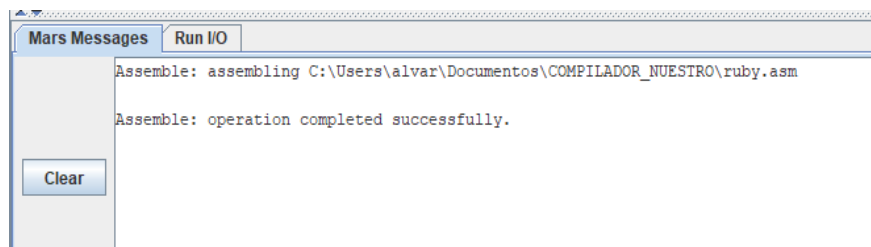
PUTS
NODO_TIPO_34

=== TABLA DE SIMBOLOS ===
Nombre      Tipo      Valor      Registro
=====
b            integer    0           $t0
a            integer    0           $t1
n            integer    0           $t2
resultado1   integer    0           $t3
resultado2   integer    0           $t0

[COMPILACION FINALIZADA]
Compilacion exitosa. Codigo MIPS generado en ruby.asm

```

Tras esto, nos vamos al MARS, abrimos el archivo "ruby.asm" tal y como hemos explicado en el apartado anterior(4), lo compilamos, y vemos como compila exitosamente:



6.Comentarios y conclusiones

Problema 1: Variables Duplicadas en MIPS

Desafío: El generador de código MIPS creaba declaraciones duplicadas de variables

Solución: Implementación de sistema de seguimiento para evitar duplicados y tomar el último valor asignado

Problema 2: Compatibilidad Windows/MinGW

Desafío: Los comandos del Makefile no funcionaban correctamente en entorno Windows

Solución: Adaptación del Makefile con comandos específicos de Windows y rutas correctas

Problema 3: Gestión de Registros MIPS

Desafío: Asignación eficiente de registros temporales en código generado

Solución: Estrategia simple pero efectiva usando registros \$t0-\$t7 con reutilización

Problema 4: Condiciones de Bucles y Condicionales

Desafío: Evaluación correcta de condiciones en estructuras de control

Solución: Generación de etiquetas apropiadas y saltos condicionales correctos

En general, el compilador Ruby a MIPS desarrollado representa un logro significativo que demuestra la viabilidad de crear herramientas de compilación funcionales utilizando metodologías establecidas.

Éxito Técnico: El compilador traduce correctamente código Ruby a MIPS ejecutable, manejando todas las características básicas del lenguaje.

Valor Académico: El proyecto proporciona experiencia práctica invaluable en todos los aspectos de construcción de compiladores, desde la teoría hasta la implementación práctica.

Este proyecto ha consolidado el entendimiento de los principios fundamentales de compiladores y ha demostrado que con las herramientas adecuadas y una metodología sólida, es posible desarrollar software de sistemas complejos y funcionales.